

Introduction

En astrophysique, la détection d'exoplanète est un enjeu important, puisqu'elle permet de détecter des planètes à l'extérieur du système solaire. La méthode la plus utilisée est la méthode du transit. Celle-ci permet de détecter la présence d'exoplanètes grâce à une diminution temporaire du flux lumineux observé d'une étoile. Un problème survient alors. Comment bien classer, et ce dans un temps raisonnable, les milliers de courbes de flux que nous transmettent les télescopes ? Par exemple, en 9 ans de service, le télescope Kepler a observé 530 506 étoiles et a découvert 2 662 exoplanètes[1] et analyser toutes les courbes de flux prendraient beaucoup trop de temps pour des résultats qui sont pour la très grande majorité des résultats négatifs.

Dans ce projet, nous cherchons à entraîner un réseau de neurones convolutif (CNN) en 1D afin de classer des courbes de flux en fonction de la présence ou non d'un transit d'exoplanète. L'idée est de prendre des données publiques du télescope Kepler pour entraîner et valider le modèle. Cela permettra de savoir si un CNN 1D est capable de classer efficacement des courbes de flux selon la présence ou l'absence d'un transit d'exoplanète. Le CNN a été choisi pour sa capacité à extraire des motifs locaux dans une série temporelle, ce qui est utile pour détecter une baisse de flux liée à un transit.

Méthodologie

Données utilisées

Les données utilisées proviennent d'un jeu de données de la campagne 3 de la mission K2 du télescope Kepler, qui ont été analysées au préalable pour donner une étiquette sur chacune des courbes [2] : 2 pour une courbe indiquant la présence d'une ou plusieurs exoplanètes et 1 pour une courbe sans exoplanète détectée. Les courbes sont représentées par une série de valeur de flux et sont fournies sous forme de dataset, ce qui rend l'extraction et l'utilisation très facile.

Les données sont ensuite chargées à l'aide d'une classe personnalisée `ExoplanetDataset`. Cette classe lit un fichier CSV contenant une colonne `LABEL` et plusieurs colonnes de flux, identifiées par le préfixe `FLUX`. Les étiquettes sont converties afin d'obtenir une classification binaire, où 0 correspond à une courbe sans exoplanète et 1 à une courbe avec exoplanète. Chaque courbe de flux est ensuite convertie en tenseur PyTorch pour être utilisée par le réseau de neurones.

Prétraitement

Un prétraitement des données a été effectué aux courbes de flux avant de pouvoir les envoyer au modèle pour l'entraînement.

Tout d'abord, les maximums extrêmes sont remplacés par la moyenne de la courbe pour limiter l'effet des valeurs extrêmes positives. Ce processus est répété un nombre arbitraire d'itérations. Dans notre cas, nous avons décidé d'effectuer trois itérations de correction de valeurs extrêmes pour nous assurer d'uniquement traiter les valeurs aberrantes.

Par la suite, un suréchantillonnage aléatoire des courbes possédant un transit a été effectué dans l'ensemble d'entraînement afin de réduire le déséquilibre entre les 2 classes. En effet, l'ensemble d'entraînement possède 5050 courbes sans exoplanète et seulement 37 avec exoplanète. Cette suréchantillonnage a été réalisé avec la fonction `RandomOverSampler` de la librairie `imblearn`. Il est important de souligner que l'ensemble de test n'a pas été suréchantillonné pour préserver le ratio des deux classes et ainsi conserver une évaluation représentative des données originales.

Modèle utilisé

Le modèle utilisé est un réseau de neurones convolutif unidimensionnel. La première partie contient des couches de convolution 1D, qui servent à récupérer les motifs locaux des courbes. Ces motifs peuvent correspondre à des variations brusques, à des diminutions de flux ou à motifs récurrents dans la courbe.

Après chaque couche de convolution, nous appliquons une normalisation par lot (Batch-Norm) afin de stabiliser et accélérer l'entraînement en ajustant les entrées de chaque couche.

Après la normalisation, nous appliquons une fonction d'activation ELU, ce qui permet d'introduire de la non-linéarité dans le modèle. Étant donné que la convolution peut être représentée comme une multiplication matricielle, l'opération est linéaire et ne peut donc, que modéliser des opérations linéaires. En introduisant des fonctions d'activations, nous venons introduire de la non-linéarité dans les opérations, ce qui permet au modèle d'apprendre sur des motifs non linéaires.

À la sortie des premières et deuxièmes couches de convolution, nous ajoutons un `MaxPool1D` qui réduit la taille du vecteur de données en ne sélectionnant que les sorties les plus hautes des couches de convolution. Puisque nous utilisons un `MaxPool` de 2, nous prenons la sortie la plus haute pour chaque pair de données consécutives, ce qui réduit la taille des données par un facteur 2.

La sortie du CNN est ensuite aplatie et envoyée dans un perceptron multicouche. Celui-

ci sert de classificateur et produit un logit, qui est ensuite transformé, avec une fonction sigmoïde, en probabilité de présence d'exoplanète. La fonction sigmoïde est donnée par l'équation suivante :

$$p(x) = \frac{1}{1 + e^{-x}}, \quad (1)$$

où x est le logits. Dans le MLP, nous voyons l'apparition de couches de **Dropout**. Pendant l'entraînement, le dropout déconnecte un pourcentage de neurones, ce qui a pour effet de forcer les neurones à apprendre des motifs et à ne pas codépendre d'un autre neurone. La figure ci-dessous donne une représentation visuelle de mon modèle :

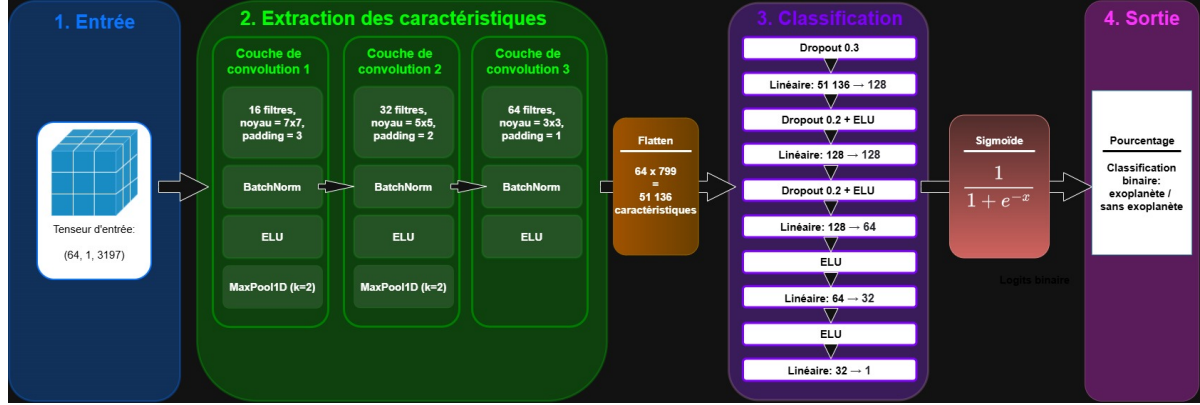


FIGURE 1 – Schéma de la structure du réseau de neurones

Entraînement

Pour entraîner le modèle, nous avons décidé d'utiliser l'optimiseur AdamW. Cet algorithme Adam permet un ajustement individuel des taux d'apprentissage de chaque paramètre en prenant en compte le gradient des itérations d'entraînement antérieures. Ce qui sépare AdamW à Adam est la capacité d'AdamW de découpler la régularisation des poids, c'est-à-dire un ajout de pénalité sur les poids trop grand, ce qui a pour effet d'améliorer la capacité de généralisation du modèle, parfois au détriment de l'erreur d'entraînement. Ces deux facteurs permettent au modèle de converger efficacement vers un minimum de la fonction de perte. Voici le fonctionnement de l'optimiseur Adam qui fonctionne comme A [3] :

Algorithm 2 Adam with L₂ regularization and Adam with decoupled weight decay (AdamW)

```

1: given  $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $m_{t=0} \leftarrow \mathbf{0}$ , second moment vector  $v_{t=0} \leftarrow \mathbf{0}$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$  ▷ select batch and return the corresponding gradient
6:    $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  ▷ here and below all operations are element-wise
8:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
9:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  ▷  $\beta_1$  is taken to the power of  $t$ 
10:   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  ▷  $\beta_2$  is taken to the power of  $t$ 
11:   $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$  ▷ can be fixed, decay, or also be used for warm restarts
12:   $\theta_t \leftarrow \theta_{t-1} - \eta_t \left( \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_{t-1} \right)$ 
13: until stopping criterion is met
14: return optimized parameters  $\theta_t$ 

```

FIGURE 2 – Comparaison d’Adam avec régularisation et AdamW

Ce que l’optimiseur cherche à minimiser est la fonction de perte qui représente la divergence entre les prédictions du modèle et les résultats réels. Puisque le modèle fait une classification binaire, nous avons décidé la fonction de perte Binary Cross Entropy (BCE) qui quantifie la différence entre les étiquettes (0 et 1) et la prédiction du modèle et est donnée par :

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)], \quad (2)$$

où N est le nombre total d’observations, y_i est l’étiquette réelle de la i ème observation et p_i est la probabilité prédite de la i ème observation.

En plus de ces deux paramètres, nous utilisons le *learning rate scheduler* `ReduceLROnPlateau` qui baisse le taux d’apprentissage lorsque l’erreur de validation stagne pour un nombre d’époques (une itération de toutes les données d’entraînement) que nous choisissons. Pour notre cas, nous avons pris trois époques qui permettent de réduire rapidement le taux d’apprentissage sans qu’il baisse trop vite si le modèle passe une époque peu fructueuse. Pour éviter le surapprentissage, le early stopping (arrêt hâtif) permet d’arrêter l’entraînement lorsque la perte de validation arrête de diminuer, ce qui indique que le modèle a atteint le meilleur entraînement possible et que continuer serait nuisible à la généralisation du modèle.

Pour entraîner le modèle, nous envoyons un lot de 64 courbes au modèle. Celui-ci nous sort ses prédictions que nous utilisons pour calculer la perte avec BCE. Pour s’assurer que l’optimiseur n’est pas affecté par les itérations antérieures, on remet ses gradients à zéro. Après la réinitialisation, on fait une rétropropagation et on met à jour les paramètres.

Métriques d'évaluation

Pour évaluer le modèle, nous nous servons de la matrice de confusion. Celle-ci permet de visualiser la performance en affichant les prédictions du modèle et pour une classification binaire, la matrice est composée de quatre catégories : les vrais positifs (VP - Réel=Positif et Prédiction=Positif), les vrais négatifs (VN - Réel=Négatif et Prédiction=Négatif), les faux positifs (FP - Réel=Négatif et Prédiction=Positif) et les faux négatifs (FN - Réel=Positif et Prédiction=Négatif). Voici à quoi ressemble la matrice de confusion :

$$\begin{bmatrix} VP & FN \\ FP & VN \end{bmatrix}$$

Les éléments de la matrice permettent de calculer les métriques d'évaluation de modèle

Exactitude

L'exactitude représente la fraction de transit bien identifié :

$$Exactitude = \frac{VP + VN}{VP + FN + FP + VN} \quad (3)$$

Précision

La précision quantifie les transits correctement identifiés parmi toutes les prédictions positives :

$$précision = \frac{VP}{VP + FP} \quad (4)$$

Rappel

Le rappel est la fraction de transits que le modèle arrive à bien identifier parmi tous les vrais transits :

$$rappel = \frac{VP}{VP + FN} \quad (5)$$

Score F1

Le score F1 est la moyenne harmonique du rappel et de la précision et permet de voir la capacité à correctement classer les courbes selon la présence ou l'absence de transit :

$$F\text{-mesure} = 2 \times \frac{précision \times rappel}{précision + rappel} \in [0, 1] \quad (6)$$

Plus le score est proche de 1, plus le modèle est performant.

Validation

Plusieurs vérifications ont été effectuées afin de valider le code. Premièrement, les dimensions des tenseurs ont été vérifiées pour s'assurer que les courbes de flux étaient bien compatibles avec les couches de convolution 1D. Plus précisément, un lot d'entraînement doit être une matrice de taille 64x3197 et nous devons avoir 64 étiquettes. Lorsque nous affichons les dimensions des lots, nous obtenons :

```
Dimensions de X\_batch : torch.Size([64, 3197])
Dimensions de y\_batch : torch.Size([64]),
```

ce qui nous indique que les lots sont de la bonne taille. Deuxièmement, les étiquettes ont été vérifiées afin de confirmer que le problème était bien une classification binaire en affichant le nombre de courbes avec l'étiquette 0 et 1 avant et après le suréchantillonnage. Nous obtenons les résultats suivants :

```
SANS OVERSAMPLING
Données d'entraînement:
-----
Nombre de courbe avec le label 0: 5050
Nombre de courbe avec le label 1: 37
-----
```

```
Données de test:
-----
Nombre de courbe avec le label 0: 565
Nombre de courbe avec le label 1: 5
-----
```

```
AVEC OVERSAMPLING
Données d'entraînement:
-----
Nombre de courbe avec le label 0: 5050
Nombre de courbe avec le label 1: 5050
-----
```

```
Données de test:
-----
Nombre de courbe avec le label 0: 565
Nombre de courbe avec le label 1: 5
-----
```

Ces résultats montrent que le changement des étiquettes des ensembles originaux (1 et 2) vers les étiquettes de classification binaire (0 et 1) a été bien effectué. De plus, nous

voyons aussi que le suréchantillonnage a bien été fait, puisqu'après celui-ci, nous avons le même nombre de courbes d'étiquette 0 que 1.

Ensuite, des courbes avec et sans exoplanète ont été affichées afin de vérifier visuellement les données fournies au modèle. Cette étape permet de confirmer que les courbes sont correctement chargées et que le prétraitement ne détruit pas complètement la structure du signal. Voici un exemple de courbes après le traitement :

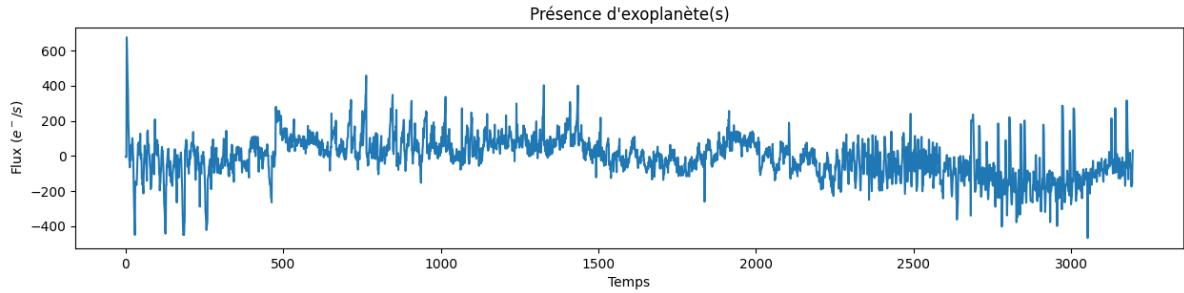


FIGURE 3 – Courbe flux avec exoplanète

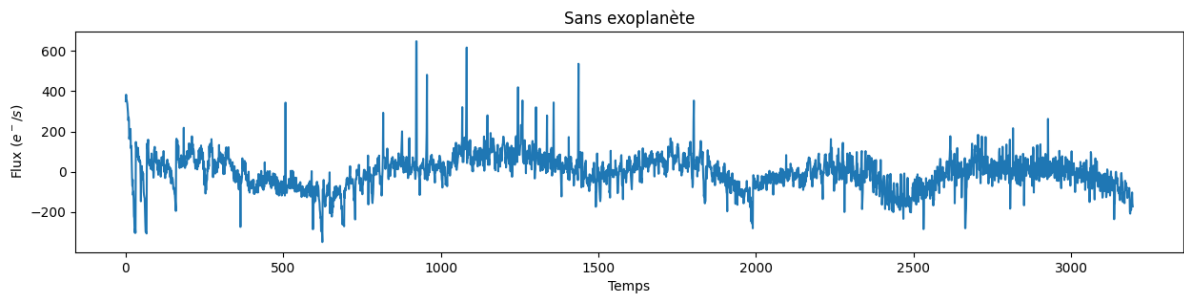


FIGURE 4 – Courbe de flux sans exoplanète

Finalement, le modèle est évalué sur un ensemble de test avec des données séparées de l'ensemble d'entraînement. Cette évaluation permet de valider le fonctionnement de l'entraînement grâce aux métriques issues de la matrice de confusion. En particulier, le score F1 est surtout utilisé puisqu'il quantifie la performance globale de modèle. Cependant, puisque les métriques seront présentées à l'analyse, nous ne montrerons pas de figure maintenant pour éviter la redondance.

Résultat

L'entraînement a été lancé sur 50 époques, mais l'arrêt hâtif a interrompu l'entraînement à l'époque 13. Le modèle avec la perte de validation la plus basse a été sauvegardé à l'époque 5 et le modèle avec le meilleur score F1 a été aussi sauvegardé à la 5e époque. Voici les graphiques de l'évolution de la perte :

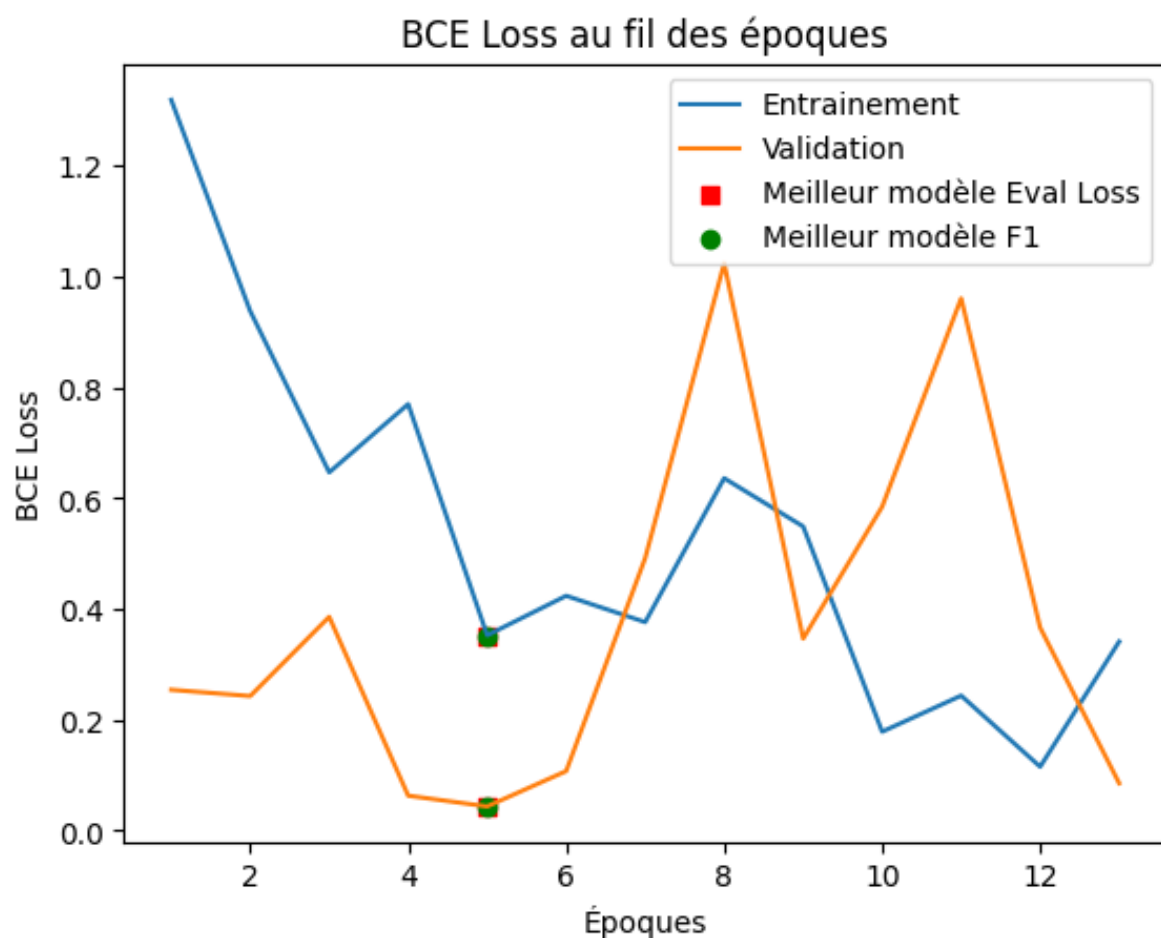


FIGURE 5 – Perte en fonction de l'époque

La figure ci-dessus montre une diminution de l'erreur d'entraînement au fil du temps, cependant, l'erreur de validation semble très instable après la cinquième époque. Cela pourrait s'expliquer soit par une atteinte très rapide du surapprentissage du modèle, soit par une instabilité liée au déséquilibre des classes. En effet, étant donné que nous avons 565 courbes sans transit et seulement 5 courbes avec transit, une erreur vaut extrêmement cher et augmente drastiquement l'erreur de validation. Ce graphique ne nous permet toujours pas de savoir la performance du modèle dans les données de validations et c'est pourquoi la figure 6 montre l'évolution des quatre métriques en fonction des époques :

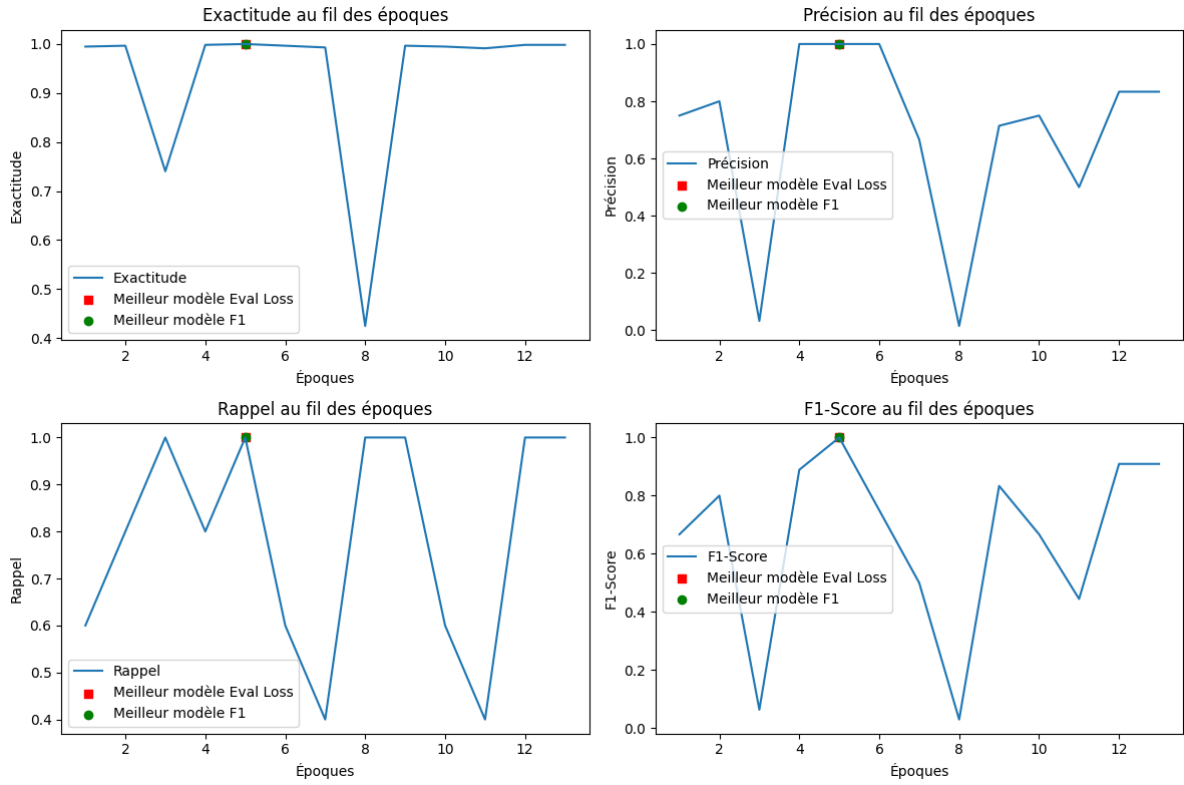
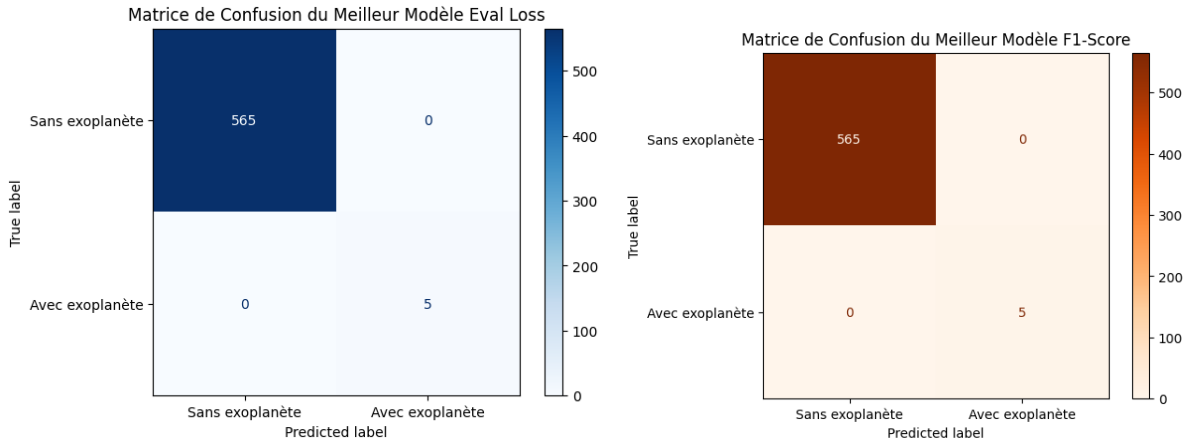


FIGURE 6 – Métriques en fonction des époques

Comme pour la figure 5, les courbes sont très instables pour les mêmes raisons que pour la courbe de perte de validation, puisque la matrice de confusion est créée à partir des résultats de l'évaluation du modèle sur les données de test. Par contre, nous voyons que les deux règles de sélection de modèles choisissent le même point qui est la valeur maximale du score F1. Nous pouvons voir les matrices de confusion à ce point pour voir la performance du modèle :



(a) Matrice de confusion du meilleur modèle selon la perte de validation (b) Matrice de confusion du meilleur modèle selon la perte de validation

FIGURE 7 – Matrice de confusion selon deux méthodes de sélection

Afin d'évaluer la stabilité du modèle, 20 entraînements indépendants ont aussi été fait. Les métriques obtenues à chaque entraînement ont été regroupées sous forme d'histogramme. Cette approche permet de vérifier si les performances sont reproductibles ou si elles dépendent de l'initialisation aléatoire des poids du réseau. La figure 8 montre ces résultats.

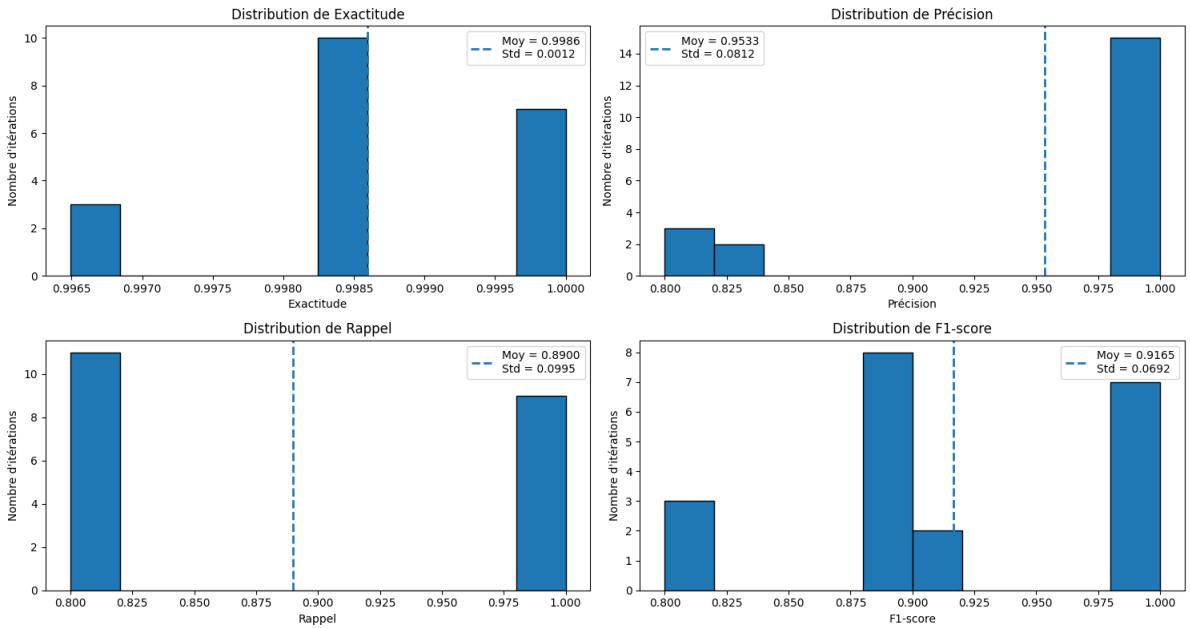


FIGURE 8 – Distribution des métriques pour 20 entraînements indépendants de modèle. Puisque la métrique F1 mesure la performance globale du modèle, la distribution sera celle pour les modèles choisis par leur score F1.

Analyse

La figure 7 montre qu'il est possible d'entraîner un modèle capable de classer efficacement les courbes de flux du jeu de données utilisé. Les performances parfaites indiquent que le modèle a appris des caractéristiques vitales permettant de séparer les courbes avec transit des courbes sans transit. Par exemple, les modèles "parfaits" arrivent à correctement classer des courbes que nous pourrions considérer comme étant des faux positifs. Voici un exemple flagrant de ce qui pourrait être considéré comme étant un faux positif :

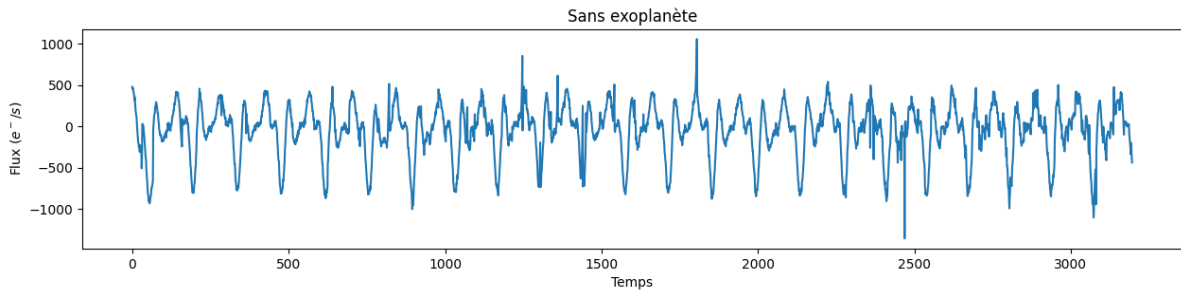


FIGURE 9 – Courbe de flux possédant des caractéristiques de transit d'exoplanète, mais ne possédant pas d'exoplanète

Cependant, ce score parfait n'est pas stable, comme le montre la figure 8 où est-ce qu'on voit que le modèle parfait n'est pas le modèle dominant parmi les 20 modèles entraînés. Cela nous pousse à penser que le score du modèle dépend de l'initialisation des poids au début de l'entraînement.

De plus, nous voyons aussi qu'avec deux critères de sélection, nous réussissons à sélectionner le même modèle deux fois. Cependant, dû à la nature des critères de sélection, il est possible d'avoir deux modèles avec des points différents. Étant donné que le critère F1 choisit le modèle selon son score F1, il va plus se concentrer sur les métriques de l'évaluation, ce qui fait en sorte que le critère F1 dépend fortement du seuil de classification que nous fixons. Par exemple, notre seuil se situe à 0.5, mais il est tout à fait possible de le modifier, ce qui aurait pour conséquence de changer drastiquement les prédictions du modèle. De son côté, le critère de sélection par perte de validation va plutôt se fier à la confiance du modèle dans ses prédictions et les comparer avec les résultats réels. La perte de validation est donc souvent mieux pour suivre l'évolution de l'apprentissage, alors que le score F1 est mieux pour sélectionner la meilleure version du modèle selon sa performance à l'évaluation.

Une limite du projet est que les résultats sont obtenus sur un jeu de données déjà analysé et étiqueté. De plus, toutes les données proviennent du même télescope, ce qui a pour effet de rendre le modèle très bon, mais uniquement pour un télescope. Si nous voulons réellement tester la capacité de généralisation du modèle sur des données provenant d'autre source, comme TESS ou mêmes de Kepler avant la mission K2 qui fournissait

des images différentes que celles de la mission K2. De manière générale, les résultats seront inférieurs à ce que nous avons produit, puisque le modèle ne sera jamais capable de généraliser les données de TESS, puisque les paramètres d'observation, comme les instruments, le bruit ou les durées d'observation peuvent être différents.

Conclusion

Ce projet avait pour objectif d'entraîner un réseau de neurones convolutif unidimensionnel afin de détecter la présence d'exoplanètes dans des courbes de flux issues de la mission K2. Pour cela, les courbes ont été prétraitées, puis utilisées pour entraîner un modèle de classification binaire en PyTorch.

Les performances du modèle ont été évaluées à l'aide de la matrice de confusion, de l'exactitude, de la précision, du rappel et du score F1.

Les résultats obtenus montrent que le CNN est capable de bien classer les courbes de flux du jeu de données utilisé. Le modèle atteint de très bonnes performances sur l'ensemble de test, ce qui indique qu'il arrive à repérer les motifs associés à la présence d'un transit. Cependant, ces résultats doivent être interprétés avec prudence. Le modèle parfait n'est pas une solution stable et les résultats ne garantissent pas nécessairement une généralisation parfaite à d'autres observations, comme celles provenant d'autres campagnes ou d'autres télescopes.

Une amélioration possible serait donc de tester le modèle sur des données plus variées, par exemple provenant de TESS ou d'autres campagnes K2. Il serait aussi intéressant d'étudier plus en détail l'effet du prétraitement et du suréchantillonnage sur les performances finales.

Références

- [1] Wikipédia, l'encyclopédie libre. *Kepler (téléscope spatial)*. (2025, novembre 2). Page consultée le 14 :49, novembre 2, 2025 à partir de https://en.wikipedia.org/w/index.php?title=Kepler_space_telescope&oldid=1349708933
- [2] Dataset Kaggle, *Kepler Labelled Time Series Data*. <https://www.kaggle.com/datasets/keplersmachines/kepler-labelled-time-series-data/data?select=exoTrain.csv>
- [3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. The MIT press. <http://www.deeplearningbook.org>
- [4] Loshchilov, I., & Hutter, F. (2019). *Decoupled weight decay regularization*(arXiv :1711.05101). arXiv. <https://doi.org/10.48550/arXiv.1711.05101>
- [5] Álvarez, S. I., Alonso, E. D., Sánchez, M. L., Rodríguez, J. R., Lasheras, F. S., & Juez, F. J. de C. (2023). *One-dimensional Convolutional Neural Networks for*

- Detecting Transiting Exoplanets* (arXiv :2312.07161). arXiv. <https://doi.org/10.48550/arXiv.2312.07161>
- [6] Wang, J. (2025). *Training a convolutional neural network for exoplanet classification with transit photometry data*. *Scientific Reports* 15(1), 15408. <https://doi.org/10.1038/s41598-025-98935-8>
 - [7] ChamCarthy, K. (2025). *Advanced Modeling for Exoplanet Detection and Characterization* (arXiv :2506.17665). arXiv. <https://doi.org/10.48550/arXiv.2506.17665>
 - [8] Fiscale, S., Ciaramella, A., Inno, L., Covone, G., Ferone, A., Rotundi, A., Hoffman, K. L., Quintana, E., Rowe, J. F., Bifulco, I., Cacciapuoti, L., Gallo, F., & Ienco, R. (2021). *Exploiting Kepler’s Heritage : A Transfer Learning Approach for Identifying Exoplanets’ Transits in TESS Data*. *Research Notes of the AAS*, 5(4), 91. <https://doi.org/10.3847/2515-5172/abf56b>